Kelvin Del Castillo | Jesús Almonte | Wesley Reyes
Roberto McKinney | Eduardo Volquez | Kellyn Santana

Tarea 1 – Investigación.

a. Fases para resolver un problema computable:

Resolver un problema en la computadora es como seguir una receta de cocina.

Paso 1: Entender el Problema. En esta fase, primero entendemos bien cuál es el problema que queremos resolver. Imaginamos que es como conocer todos los ingredientes antes de cocinar. Necesitamos saber qué se necesita, qué restricciones hay y qué espera la persona que usará el programa. Esto puede implicar hacer preguntas, investigar y recopilar información útil.

Paso 2: Diseñar la Solución (Algoritmo). Una vez que sabemos de qué se trata el problema, es hora de diseñar la solución. Es como planear cómo cocinar una comida. Creamos un plan, una especie de receta, que nos dice cómo vamos a resolver el problema. A veces, usamos dibujos o esquemas para visualizar esto, lo importante es tener una lluvia de ideas.

Paso 3: Hacer la Receta en Código. Aquí es donde traducimos nuestra receta en un lenguaje que la computadora pueda entender. Es como seguir los pasos para cocinar. Escribimos las instrucciones exactas que la computadora debe seguir para resolver el problema. Mantenemos el código ordenado y claro para que todos lo entiendan.

Paso 4: Probar y Ajustar. Después de escribir el código, lo probamos como si probáramos la comida. Compilamos el código, lo ejecutamos y vemos si funciona como debería. Si algo no está bien, buscamos errores y los corregimos hasta que todo funcione sin problemas.

Paso 5: Documentación. Al igual que anotar la receta de cocina, dejando notas y mensajes para mejorar una próxima ejecución o preparación, creamos documentación para explicar cómo funciona nuestro programa. Esto ayuda a otras personas a entenderlo. La documentación puede estar en el código mismo o en un manual externo.

Paso 6: Mantenimiento. Una vez que el programa está en uso, no termina ahí. Es como cuidar de un jardín. Hay que mantenerlo actualizado, corregir errores que surgen con el tiempo y adaptarlo a nuevos cambios si es necesario.

b. Partes y tipos de algoritmos computacionales:

Los algoritmos computacionales son fundamentales en la programación y la informática en general, ya que proporcionan un camino ordenado y paso a paso para solucionar problemas.

En términos sencillos, un algoritmo se divide en dos partes:

1. La Entrada: Aquí es donde definimos los datos o información que necesitamos para que el algoritmo haga su trabajo. Puede ser números, palabras, imágenes, todo dato que sea necesario para que funcione correctamente.

2. El Proceso: Esta es la parte principal del algoritmo. Aquí es donde describimos las instrucciones específicas y la lógica que la computadora debe seguir para resolver el problema o realizar la tarea. Esto puede incluir matemáticas, comparaciones, bucles para repetir acciones y decisiones basadas en condiciones.

Los algoritmos se clasifican en diferentes tipos según lo que hagan:

1. Secuenciales: Son como una receta en la que sigues pasos uno tras otro en orden. Son útiles para tareas simples y lineales.

2. Condicionales: Estos algoritmos toman decisiones basadas en ciertas condiciones. Dependiendo de lo que esté sucediendo, el algoritmo puede tomar diferentes caminos.

3. Bucle (Iterativos): Repiten acciones hasta que se cumple una condición específica. Son buenos para hacer cosas repetitivas o trabajar con listas de información.

4. Recursivos: En lugar de usar bucles, estos algoritmos se llaman a sí mismos para resolver un problema dividiéndolo en partes más pequeñas y manejables. Son útiles para problemas matemáticos y de búsqueda.

5. Clasificación y Búsqueda: Se usan para organizar información en orden o encontrar cosas específicas en una lista.

6. Grafos y Árboles: Resuelven problemas relacionados con la teoría de grafos, como encontrar la ruta más corta en una red o ver cómo se conectan diferentes puntos en un grafo.

7. Algoritmos de Optimización: Buscan encontrar la mejor solución entre un conjunto de posibles soluciones. Ejemplos incluyen algoritmos genéticos y algoritmos de optimización por enjambre de partículas.

8. Algoritmos Criptográficos: Utilizados en seguridad informática y criptografía para proteger la información mediante técnicas de cifrado y descifrado.

9. Algoritmos de Divide y Vencerás: Estos algoritmos resuelven problemas dividiéndolos en subproblemas más pequeños, resolviendo cada subproblema y luego combinando las soluciones de los subproblemas para obtener la solución del problema original.

10. Algoritmos Greedy (Voraces): Estos algoritmos toman decisiones locales óptimas en cada paso con la esperanza de encontrar una solución global óptima. A menudo se usan en problemas de optimización.

11. Algoritmos de Programación Dinámica: Estos algoritmos resuelven problemas dividiéndolos en subproblemas, pero a diferencia de los algoritmos de Divide y Vencerás, los resultados de los subproblemas se almacenan y reutilizan para evitar redundancia en los cálculos.

12. Algoritmos de Retroceso (Backtracking): Estos algoritmos exploran todas las posibles soluciones a un problema evaluando cada opción en un intento de encontrar la solución. Si una opción no conduce a la solución, el algoritmo retrocede y prueba una alternativa.

La elección de un tipo de algoritmo depende de lo que necesites hacer y de cómo quieras hacerlo. Los programadores eligen el algoritmo adecuado para resolver una amplia variedad de problemas, desde trabajar con datos hasta hacer que las computadoras sean más inteligentes y eficientes.

c. Historia de los algoritmos

Antes de adentrarnos en la historia de los algoritmos, es importante entender qué significa esta palabra en términos sencillos. Un algoritmo es, en pocas palabras, un conjunto de pasos que seguimos para realizar una tarea, ya sea algo tan cotidiano como hablar por teléfono con tu mamá o llevar a cabo tareas en tu computadora. Aunque en el pasado la gente utilizaba algoritmos sin tener una palabra específica para describirlos, hoy en día es más claro lo que son. Un ejemplo común de algoritmo es hacer operaciones matemáticas como sumar, restar, multiplicar, dividir, o calcular raíces cuadradas, hipotenusas y otros cálculos matemáticos.

El término "algoritmo" se le atribuye a Khowârizmî, quien fue uno de los primeros en utilizar esta palabra. En sus escritos, empleó el sistema de numeración hindú y el concepto del cero. Esto representó un gran avance, ya que nos enseñó cómo llevar a cabo cálculos matemáticos paso a paso, de forma manual. Hoy en día, disponemos de calculadoras en nuestros teléfonos y computadoras, lo que hace que sea mucho más sencillo obtener resultados de manera rápida. A pesar de que los escritos originales de Khowârizmî se han perdido, todavía tenemos una versión en latín de sus ideas.

Es importante destacar que Khowârizmî no fue el inventor de los algoritmos, pero sí fue uno de los primeros en pensar de manera algorítmica. Los algoritmos existían mucho antes de que él los describiera. En el pasado, el término "algoritmo" se aplicaba principalmente a las matemáticas, pero con el tiempo, comenzó a utilizarse en situaciones cotidianas, especialmente desde el siglo XVIII hasta la actualidad.

Charles Babbage, quien inventó la máquina analítica, se considera uno de los pioneros en el desarrollo de los "algoritmos digitales" a mediados del siglo XIX. Esto marcó un gran avance tecnológico, ya que permitió la digitalización de procesos que antes se realizaban de forma manual, lo que hizo más eficiente la obtención de los algoritmos deseados. Con el tiempo, los matemáticos

comenzaron a preguntarse si sería posible encontrar una forma sencilla de determinar si un problema matemático tiene solución.

Estas preguntas se fueron resolviendo gradualmente con el tiempo. En 1980, se produjo una revolución en la computación cuántica que superó la limitación de operar solo con valores 0 o 1, una limitación de los algoritmos clásicos. Esto fue posible gracias al trabajo de dos personas destacadas, Paul Benioff y Richard Feynman. Una década después, los científicos comenzaron a desarrollar algoritmos específicos para las computadoras cuánticas de esa época. Esto marcó un nuevo avance en la tecnología de los algoritmos.

d. Importancia de los algoritmos

Los algoritmos son como las herramientas mágicas que hacen que la tecnología funcione y nos ayudan a resolver problemas en todos los aspectos de la vida moderna. Son la clave de todo el proceso detrás de las pantallas. Aquí te compartimos por qué son tan importantes:

1. Resolver problemas: Los algoritmos son como un GPS para solucionar problemas, ya sea hacer cuentas complicadas o encontrar el camino más rápido para entregar paquetes o buscar información en una montaña de datos.

2. Automatización: Los algoritmos son como tus asistentes personales que hacen trabajos repetitivos o complicados sin cansarse. Esto mejora la velocidad y reduce errores en cosas como la producción y manejo de información.

3. Inteligencia Artificial: Los algoritmos son como el cerebro detrás de la inteligencia artificial. Ayudan a las máquinas a aprender, tomar decisiones y volverse más inteligentes con el tiempo.

4. Ciencia de Datos: En el mundo de los datos, los algoritmos son como los detectives que encuentran patrones ocultos en grandes montañas de información. Esto es útil en el análisis de mercados, medicina y muchas otras áreas.

5. Seguridad en línea: Los algoritmos también son guardianes digitales. Protegen tus datos personales y transacciones en línea mediante cifrado y medidas de seguridad.

6. Entretenimiento: Los algoritmos son la magia detrás de los videojuegos y las películas. Crean efectos especiales y experiencias interactivas.

7. Optimización: Los algoritmos son como consejeros expertos que te ayudan a tomar las mejores decisiones en situaciones complicadas, como planificar rutas, asignar recursos o programar proyectos.

8. Búsqueda en Internet: Si usas motores de búsqueda como Google, debes agradecer a los algoritmos por encontrar la información que necesitas en medio de un océano de páginas web.

9. Medicina: Los algoritmos también son héroes en el campo de la medicina. Ayudan a diagnosticar enfermedades, analizar imágenes médicas, predecir riesgos y diseñar tratamientos personalizados.

10. Economía y Finanzas: Los algoritmos tienen un papel importante en las decisiones financieras, desde el análisis de inversiones hasta la gestión de carteras y el comercio en línea.

e. Funciones de Librería (Library Functions)

Las funciones de librería computacionales son como bloques de construcción ya hechos para tus programas de computadora. Imagina que son como herramientas listas para usar. Aquí te explico por qué son tan útiles:

1. Reutilización de código: Estas funciones son como recetas de cocina que ya están escritas. En lugar de empezar desde cero cada vez que necesitas hacer algo en tu programa, puedes usar estas recetas ya probadas.

2. Eficiencia: Las funciones de librería son como superhéroes de la velocidad. Están diseñadas para funcionar de manera rápida y eficiente, lo que significa que tus programas se ejecutarán más rápido.

3. Abstracción: ¿Te imaginas si cada vez que quisieras hacer un sándwich tuvieras que cultivar el trigo y hacer el pan desde cero? Las funciones de librería son como ir a la tienda y comprar el pan ya hecho. Te ahorran tiempo y esfuerzo porque no necesitas preocuparte por cómo funcionan por dentro.

4. Variedad de tareas: Estas funciones no solo hacen matemáticas, también pueden manejar textos, archivos, y un montón de otras cosas. Son como una caja de herramientas gigante para programadores.

5. Manual de instrucciones: Cada función viene con un manual de usuario. En otras palabras, hay documentación que te dice cómo usarlas. Así que si alguna vez te preguntas "¿cómo hago esto?", solo consulta el manual.

6. Facilitan la vida: En resumen, estas funciones hacen que la vida de los programadores sea más fácil. Te ahorran tiempo y evitan dolores de cabeza, porque no tienes que escribir todo desde cero cada vez.

Si pensamos en las funciones matemáticas como las que usas en una calculadora. También están las que te ayudan a manejar palabras o abrir archivos. Son herramientas útiles que hacen que la programación sea más sencilla y eficiente.